UNIVERSITY OF GHANA
SCHOOL OF ENGINEERING SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING
CPEN 421: MOBILE AND WEB SOFTWARE DESIGN AND ARCHITECTURE

FINAL COURSE PROJECT – DEADLINE: 31st MARCH 2026

INTRODUCTION

In this course project, you will design and build a distributed software system that simulates a national emergency response and dispatch coordination platform. The purpose of this project is to help you gain practical experience building a real world, microservices based system that communicate using HTTP APIs, message queues, and real-time communication technologies such as web sockets.

In many African countries, including Ghana, emergency services such as police stations, hospitals, fire service stations, and ambulance units often operate independently. One of the biggest challenges during emergencies is coordination: determining which responder is closest, which hospital has available beds, or which ambulance can respond immediately.

In this project, you will design a platform that helps coordinate emergency responses. The system will allow administrators to record emergency incidents, automatically determine the nearest appropriate responder, dispatch the unit, and track the response in real time. Your final system should demonstrate how modern distributed software systems can support critical national infrastructure such as emergency services.

SYSTEM DESCRIPTION BY STAKEHOLDER (GOVERNMENT OF GHANA)

The system will have administrators at call centers. An administrator who receives an emergency call will log into the system and record the incident. During the call the admin will briefly ask the caller questions and fill an incident reporting form containing the following information:

- Details of the citizen reporting the incident
- Type of incident
- Location of the incident
- Any additional notes about the incident

The location must be selected using Google Maps, and the system must store the latitude and longitude of the incident. Once the administrator submits the incident, the system must determine which emergency unit should respond.

Depending on the type of incident, the system must select the nearest available responder such as:

- Police station

- Fire service station
- Ambulance attached to a hospital

For medical emergencies, the system must also determine which hospital has available capacity to receive the patient. The system must then dispatch the responder and track the response in real time.

SYSTEM ACTORS

Your system should have at least the following actors:

- **System Administrators**: These are emergency operators who receive calls from citizens and record incidents in the system.
- **Hospital Administrators**: Hospital staff responsible for updating hospital capacity and ambulance availability as well as staff information if necessary.
- **Ambulance Drivers**: Drivers who operate ambulances and whose vehicles transmit location updates.
- **Police Station Administrators**: Law enforcement staff attached to police stations and responsible for adding new police officers to the system and updating information for specific police stations.
- **Fire Service Administrators**: Fire service admins that perform similar functionality as the police station and hospital administrators.
- **Citizens**: People who report emergencies via phone calls. Citizens do not log into the system.

REQUIRED MICROSERVICES

Your system **must** be implemented using microservices. Each microservice must:

- Run independently
- Have its own database
- Provide its own API endpoints
- Communicate with other services using APIs or message queues

Microservice 1: Identity and Authentication Service

This service manages all system users. Only authorized personnel should be able to access the system. Citizens do not log into the platform.

Users supported by this service include:

- System administrators
- Hospital administrators

- Police Station Administrators
- Fire Service Administrators

This service must handle:

- User registration
- Login authentication
- Role-based authorization
- Token-based authentication

You should use JWT tokens to authenticate requests between services.

Minimum API Endpoints

```
POST /auth/register
POST /auth/login
POST /auth/refresh-token
GET /auth/profile
```

Minimum Data Stored

- User ID
- Name
- Email
- Role
- Password Hash
- Created Date

Microservice 2: Emergency Incident Service

This service records and manages all emergency incidents. When an administrator receives a phone call from a citizen, the administrator will fill out an incident report form. The form must capture:

- Name of citizen reporting
- Incident type
- Location (latitude and longitude from Google Maps)
- Notes about the incident
- Administrator who created the report

After the incident is created, the system must automatically determine which emergency responder should handle the incident. For example, if the incident type is robbery or crime, select the nearest police station. If the incident type is fire, select the nearest fire service station. If the incident type is medical emergency, select the nearest available ambulance. This service must calculate the distance between the incident location and responder locations. The selected responder must be both:

- geographically closest
- currently available

The service must also track the status of the incident, such as:

1. Created
2. Dispatched
3. In Progress
4. Resolved

Minimum API Endpoints

```
POST /incidents

GET /incidents/:id

GET /incidents/open

PUT /incidents/:id/status

PUT /incidents/:id/assign
```

Minimum Data Stored

- Incident ID
- Citizen Name
- Incident Type
- Latitude
- Longitude
- Notes
- Created By (Admin ID)
- Assigned Unit
- Status
- Timestamp

Microservice 3: Dispatch Tracking Service

Ambulances, fire service trucks and police vehicles must transmit their GPS location periodically to the system. This service maintains real-time vehicle location data for every emergency response dispatched. Administrators should be able to see vehicle movement in real time. For testing, assume that the assigned responder's phone location service determines the current location of the vehicle. Example, the assigned ambulance driver's location is tracked as the location of the ambulance.

Minimum API Endpoints

```
POST /vehicles/register
GET /vehicles
GET /vehicles/:id/location
```

Minimum Data Stored

- Vehicle ID
- Hospital/Police Station/Fire Service Station ID
- Incident Service ID
- Latitude
- Longitude
- Vehicle Status

Microservice 4: Analytics and Monitoring Service

This service generates analytics and operational insights. Data from incidents, dispatch operations, hospital capacity, and vehicle tracking should be aggregated to produce useful statistics.

Examples of analytics include:

- Average response time to incidents
- Number of incidents per region per incidence type
- Hospital bed usage statistics
- Most deployed responders per emergency service, etc.

Minimum API Endpoints

```
GET /analytics/response-times
GET /analytics/incidents-by-region
GET /analytics/resource-utilization
```

PROJECT DELIVERABLES

Your project will be completed in **four phases**.

Phase 1: System Design

You must design the architecture of your system. The deliverables of this phase must include:

- Microservice architecture diagram
- Database design for each microservice
- Definition of all APIs
- Message queue definitions if any
- Event message structures

Phase 2: Backend Implementation

You must implement all microservices. Each microservice must:

- Run independently
- Have its own database
- Provide REST API endpoints
- Communicate with other services

Phase 3: Client Interface

You must implement a minimal web interface that allows administrators to:

- Log in
- Record incidents
- View dispatch status
- Track ambulance locations on a map
- View analytics related to their service

Phase 4: Documentation and Demonstration

You must submit:

- Source code for all microservices
- API documentation (Swagger or Postman)
- Deployment instructions
- A short (5 minutes max) demonstration video explaining how your system works